Computer Lab 1 Block 1

732A99 Machine Learning

Giulio Collazuol (LiU-ID), Valentín Rapetti (LiU-ID), Sebastián Soriano Pérez (sebso409)

2026-11-13

Assignment 1. Handwritten digit recognition with K-nearest neighbors

The data file `optdigits.csv` contains information about normalized bitmaps of handwritten digits, each column representing a pixel in an 8x8 image as an integer in the range from 0 to 16, with the last column containing the actual digit. We first loaded the file as a data.frame `digits`.

1. Data partitioning

The data.frame `digits` was then split into the following data.frames randomly:

- 50% `train`: 1911 rows
- 25% `valid`: 955 rows
- 25% `test`: 957 rows

For this, we used the code from Lecture 1a ("Holdout in R") and we set a seed with `set.seed(12345)`.

2. 30-nearest neighbor classifier

Table 1: Misclassification error per digit and overall (k = 30).

	0	1	2	3	4	5	6	7	8	9	All
Training	0.000	0.082	0.010	0.016	0.091	0.055	0.010	0.022	0.078	0.085	0.045
Test	0.013	0.058	0.030	0.036	0.138	0.097	0.000	0.009	0.103	0.045	0.053

As expected, we observe the train misclassification error (`0.045`) is smaller than the test misclassification error (`0.053`). The train error is slightly smaller as it is calculated on the same data that is used in the KNN model. This small gap shows little overfitting.

The digit `0` has the lowest train error at `0.000` and the third lowest test error at `0.013` after the digits `6` and `7`. The digit `6` has the lowest test error at `0.000` and is tied for the second lowest train error at `0.010` with the digit `2`. This tells us that these digits are the easiest ones to recognize in this handwritten set, considering its low resolution.

On the other hand, the digit `4` has the highest train error at `0.091` and the highest test error at `0.138`, followed by `8` at `0.103`, and `5` at `0.097`. The digit `4` is thus the hardest one to recognize in this dataset.

Table 2: Confusion matrices for the 30-nearest neighbour classifier: train (left) and test (right). Rows are the true digits, columns the predicted digits.

	0	1	2	3	4	5	6	7	8	9
0	202	0	0	0	0	0	0	0	0	0
1	0	179	11	0	0	0	0	1	1	3
2	0	1	190	0	0	0	0	1	0	0
3	0	0	0	185	0	1	0	1	0	1
4	1	3	0	0	159	0	0	7	1	4
5	0	0	0	1	0	171	0	1	0	8
6	0	2	0	0	0	0	190	0	0	0
7	0	3	0	0	0	0	0	178	1	0
8	0	10	0	2	0	0	2	0	188	2
9	1	3	0	5	2	0	0	3	3	183

	0	1	2	3	4	5	6	7	8	9
0	77	0	0	0	1	0	0	0	0	0
1	0	81	2	0	0	0	0	0	0	3
2	0	0	98	0	0	0	0	0	3	0
3	0	0	0	107	0	2	0	0	1	1
4	0	0	0	0	94	0	2	6	2	5
5	0	1	1	0	0	93	2	1	0	5
6	0	0	0	0	0	0	90	0	0	0
7	0	0	0	1	0	0	0	111	0	0
8	0	7	0	1	0	0	0	0	70	0
9	0	1	1	1	0	0	0	1	0	85

Looking at the confusion matrices, the diagonals tell us most samples are correctly classified as expected. Every other cell outside the main diagonal contains the samples that were incorrectly classified. On both the train and the test data we can see the most common error was the digit `1` being misclassified as `2`, followed closely by `8` as `1`. This could be due to the dataset's low resolution which does not allow the `8`'s details in the middle to be displayed clearly. Other common mistakes are misclassifying the digit `4` as `7` or the digit `5` as `9`. A higher resolution dataset may decrease the incidence of these errors.

In conclusion, the 30-NN classifier does a good overall job for this dataset, with a test accuracy of `94.7%`.

3. Easiest and hardest cases of digit "8"

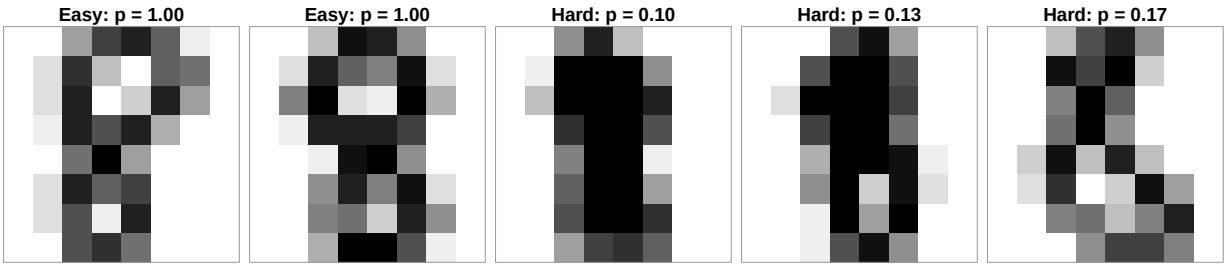


Figure 1: The two easiest (left) and three hardest (right) cases of digit 8 in the training data. p is the predicted probability of the correct class.

Here are the two easiest and the three hardest cases for the digit `8`, that is the two with the highest predicted probability and the three with the lowest. With the data resolution, we can clearly see the first two are the digit `8`, while the last three could be easily mistaken for other digits as the characteristic round figures from the digit `8` cannot be observed. These could be easily mistaken as `1` or `6` or some other digit.

In the 64-dimensional space of our features, these cases are not spatially close to the other samples of the digit `8`, and therefore their predicted probability of being `8` is low. Most of their 30 closest neighbors in this space belong to other classes.

4. Misclassification error for K = 1, . . . , 30

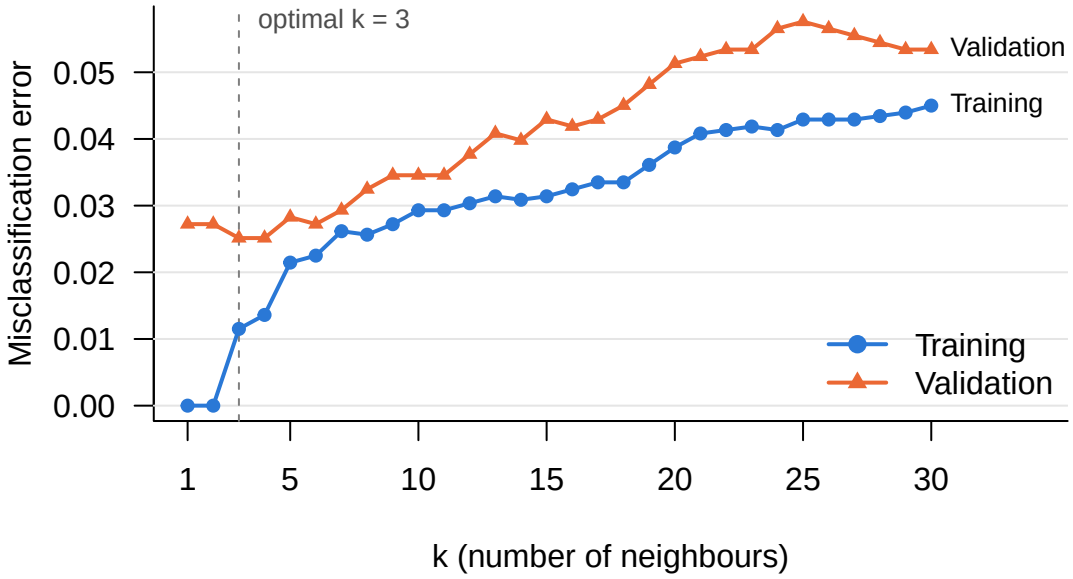


Figure 2: Training and validation misclassification error for k = 1, . . . , 30.

Figure 2 shows a small `k` returns a more flexible and complex model, while a large `k` produces a smoother and simpler model. The misclassification errors mostly increase as `k` increases. It is self-evident that using `k=1` returns an error of `0` on the training data as each observation is its own prediction. For the validation set, the error first decreases and is minimized at `k=3` and `k=4`. We are using `k=3` as it is the first one.

Table 3: Misclassification errors for the optimal k (k = 3).

Training	Validation	Test
0.0115	0.0251	0.0240

For our chosen 3-NN model, the test error is equal to `0.0240` (`97.6%` accuracy), which is very close to the validation error of `0.0251`, so the validation set was a good guide. Both of these values are considerably larger than the train error of `0.0115`, which suggests some overfitting. Figure 2 shows that at `k=7` both error lines are closest to each other, however the validation error is higher at that point. Overall, the 3-NN model has less than half the test error of the 30-NN model (`0.053`).

5. Cross-entropy for K = 1, . . . , 30

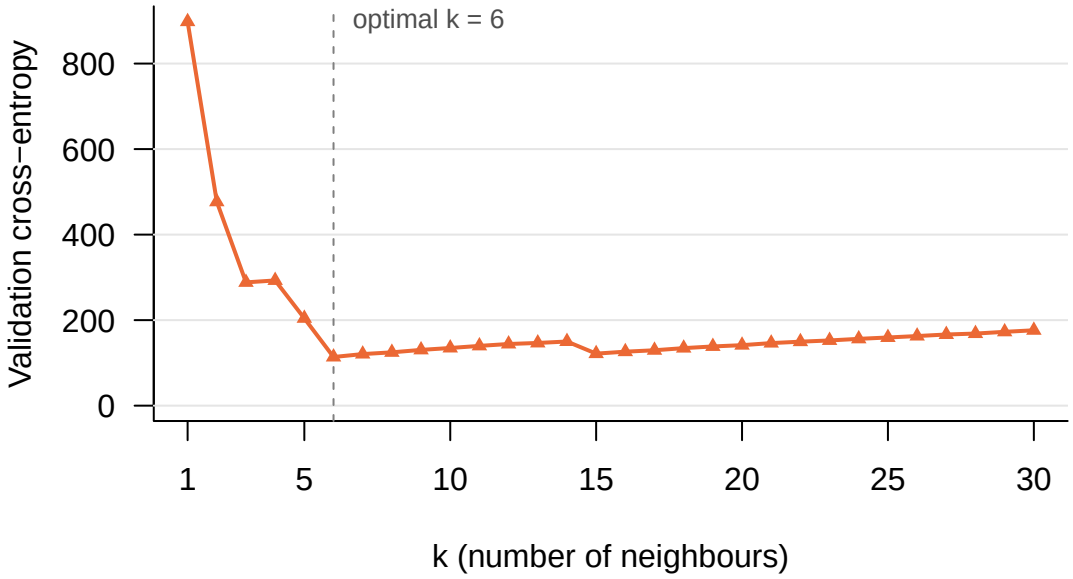


Figure 3: Validation cross-entropy for k = 1, . . . , 30.

Figure 3 shows the validation cross-entropy at different values of `k`. Here, we observe that the optimal value is `k=6`. We can observe that for small values of `k` the cross-entropy is the highest. At `k=1` every misclassified sample gets a probability `0` which adds $-log(1 \times 10^{-15}) \approx 34.5$ to the sum.

Using the cross-entropy may suit a multinomial response better because it is the minus log-likelihood of the multinomial model. It also uses the predicted probabilities instead of only using the predicted class. Thus, it rewards the model's confidence in its correct predictions and penalizes its confident incorrect predictions as well.

Assignment 2. Linear regression and ridge regression

Assignment 3. Logistic regression and basis function expansion

Assignment 4. Theory

Why can it be important to consider various probability thresholds in classification problems?

What ways of collecting correct values of the target variable for supervised learning problems are mentioned in the book?

How can one express the cost function of linear regression in matrix form?

Appendix: Code

```
library(kknn)

# optdigits.csv has no header: 64 pixel columns + last column = digit (0-9)
digits <- read.csv("data/optdigits.csv", header = FALSE)
```

```r
colnames(digits)[ncol(digits)] <- "digit"
digits$digit <- as.factor(digits$digit)
n <- dim(digits)[1]
set.seed(12345)
id_train <- sample(1:n, floor(n * 0.5))
train <- digits[id_train,]

id_tmp <- setdiff(1:n, id_train)
set.seed(12345)
id_valid <- sample(id_tmp, floor(n * 0.25))
valid <- digits[id_valid,]

id_test <- setdiff(id_tmp, id_valid)
test <- digits[id_test,]
# Fits models
fit_train <- kknn(
  digit ~ ., train = train, test = train, k = 30, kernel = "rectangular"
)
fit_test <- kknn(
  digit ~ ., train = train, test = test, k = 30, kernel = "rectangular"
)
# Gets confusion matrices
cm_train <- table(True = train$digit, Predicted = fit_train$fitted.values)
cm_test <- table(True = test$digit, Predicted = fit_test$fitted.values)

# Defines missclass function to calculate classification error
missclass <- function(true, predicted) {
  n <- length(true)
  1 - sum(diag(table(true, predicted))) / n
}

# Defines digit_missclass function to get each digit's classification error
digit_missclass <- function(true, predicted) {
  1 - diag(table(true, predicted)) / rowSums(table(true, predicted))
}

# Shows the error per digit and overall, for training and test data
error_table <- rbind(
  Training = c(
    digit_missclass(train$digit, fit_train$fitted.values),
    All = missclass(train$digit, fit_train$fitted.values)
  ),
  Test = c(
    digit_missclass(test$digit, fit_test$fitted.values),
    All = missclass(test$digit, fit_test$fitted.values)
  )
)
knitr::kable(
  error_table, digits = 3, format.args = list(nsmall = 3),
  caption = "Misclassification error per digit and overall (k = 30)."
)
# Shows the confusion matrices side by side
# (rows: true digit, columns: predicted digit).
```

```r
# The two LaTeX commands centre the tables and leave a 1 cm gap between them
knitr::kables(
  list(
    "\\centering",
    knitr::kable(cm_train),
    "\\hspace{1cm}",
    knitr::kable(cm_test)
  ),
  caption = paste(
    "Confusion matrices for the 30-nearest neighbour classifier:",
    "train (left) and test (right).",
    "Rows are the true digits, columns the predicted digits."
  )
)
# Gets the predicted probabilities for the digit 8
idx_8 <- which(train$digit == "8")
prob_8 <- fit_train$prob[idx_8, "8"]

# Gets the 2 samples of 8 with the highest probability and the 3 with the lowest
easiest <- idx_8[order(prob_8, decreasing = TRUE)[1:2]]
hardest <- idx_8[order(prob_8)[1:3]]

# Reshapes one training row into the 8x8 image (pixels are stored row by row)
plot_digit <- function(i, title) {
  img <- matrix(as.numeric(train[i, 1:64]), nrow = 8, byrow = TRUE)
  # heatmap() draws row 1 at the bottom, so reverse the rows to show the
  # digit upright; scale = "none" keeps the raw pixel counts
  heatmap(
    img[8:1,], Colv = NA, Rowv = NA, scale = "none", labRow = NA, labCol = NA,
    main = title, col = grey(seq(1, 0, length.out = 17)),
    # The frame keeps all five images at the same scale in the PDF
    add.expr = box(col = "grey60")
  )
}

for (i in easiest) {
  plot_digit(i, sprintf("Easy: p = %.2f", fit_train$prob[i, "8"]))
}
for (i in hardest) {
  plot_digit(i, sprintf("Hard: p = %.2f", fit_train$prob[i, "8"]))
}
get_knn_missclass_error <- function(train, valid, k) {
  # Fits models
  fit_train <- kknn(
    digit ~ ., train = train, test = train, k = k, kernel = "rectangular"
  )
  fit_valid <- kknn(
    digit ~ ., train = train, test = valid, k = k, kernel = "rectangular"
  )

  # Calculates train and validation error rates
  error_train <- missclass(train$digit, fit_train$fitted.values)
  error_valid <- missclass(valid$digit, fit_valid$fitted.values)
```

```
  c(error_train = error_train, error_valid = error_valid)
}

# Calculates errors for each value of k from 1 to 30
errors <- sapply(1:30, function(k) get_knn_missclass_error(train, valid, k))
k_values <- 1:30
k_opt <- which.min(errors["error_valid",])
col_train <- "#2a78d6"  # blue
col_valid <- "#eb6834"  # orange

# Plots an empty plot first so the grid and the k marker sit behind the lines
plot(
  k_values, errors["error_train",], type = "n", xaxt = "n",
  xlim = c(1, 34), ylim = c(0, max(errors)), las = 1, bty = "l",
  xlab = "k (number of neighbours)", ylab = "Misclassification error"
)
axis(1, at = c(1, seq(5, 30, by = 5)))
abline(h = axTicks(2), col = "grey90")
abline(v = k_opt, lty = 2, col = "grey50")

# Adds lines with train and validation errors
lines(
  k_values, errors["error_train",], type = "o", pch = 19, cex = 0.7,
  lwd = 2, col = col_train
)
lines(
  k_values, errors["error_valid",], type = "o", pch = 17, cex = 0.8,
  lwd = 2, col = col_valid
)

# Labels at the right end of each line, and the optimal K
text(30, errors[, 30], c("Training", "Validation"), pos = 4, cex = 0.85)
text(
  k_opt, max(errors), paste("optimal k =", k_opt), pos = 4, cex = 0.85,
  col = "grey30"
)
legend(
  "bottomright", legend = c("Training", "Validation"),
  col = c(col_train, col_valid), pch = c(19, 17), lwd = 2, bty = "n"
  )
# Sets the optimal value of k
k <- 3

# Fits a new model to get the test error
fit_test <- kknn(
  digit ~ ., train = train, test = test, k = k, kernel = "rectangular"
)

# Gets train and valid results
errors <- get_knn_missclass_error(train, valid, k)

# Shows the training, validation and test errors for the optimal k
knitr::kable(
```

```
  data.frame(
    Training = errors["error_train"],
    Validation = errors["error_valid"],
    Test = missclass(test$digit, fit_test$fitted.values)
  ),
  digits = 4, format.args = list(nsmall = 4), row.names = FALSE,
  caption = paste0(
    "Misclassification errors for the optimal k (k = ", k, ")."
  )
)
get_knn_cross_entropy <- function(train, valid, k) {
  fit_valid <- kknn(
    digit ~ ., train = train, test = valid, k = k, kernel = "rectangular"
  )
  prob <- fit_valid$prob

  # Indicator matrix I(Y_i = C_m): one row per observation, one column per digit
  Y <- outer(valid$digit, colnames(prob), "==")

  # Cross-entropy as on Lecture 1a, slide 27 (1e-15 avoids log(0))
  -sum(Y * log(prob + 1e-15))
}

cross_entropy <- sapply(
  1:30, function(k) get_knn_cross_entropy(train, valid, k)
)
k_values <- 1:30
k_opt_ce <- which.min(cross_entropy)
col_valid <- "#eb6834"  # orange, as the validation line in the Task 4 plot

# Plots an empty plot first so the grid and the k marker sit behind the line
plot(
  k_values, cross_entropy, type = "n", xaxt = "n",
  ylim = c(0, max(cross_entropy)), las = 1, bty = "l",
  xlab = "k (number of neighbours)", ylab = "Validation cross-entropy"
)
axis(1, at = c(1, seq(5, 30, by = 5)))
abline(h = axTicks(2), col = "grey90")
abline(v = k_opt_ce, lty = 2, col = "grey50")

# Adds the validation cross-entropy line
lines(
  k_values, cross_entropy, type = "o", pch = 17, cex = 0.8,
  lwd = 2, col = col_valid
)

# Marks the optimal k
text(
  k_opt_ce, max(cross_entropy), paste("optimal k =", k_opt_ce),
  pos = 4, cex = 0.85, col = "grey30"
)
# check.names = FALSE keeps the original column names, e.g. "Jitter(%)"
parkinsons <- read.csv("data/parkinsons.csv", check.names = FALSE)
```

```r
# Keep only the target and the 16 voice features
# (drops subject#, age, sex, test_time, total_UPDRS)
parkinsons <- parkinsons[, c("motor_UPDRS", colnames(parkinsons)[7:22])]
# No header; columns are in the order listed in the lab PDF
pima <- read.csv(
  "data/pima-indians-diabetes.csv",
  header = FALSE,
  col.names = c(
    "pregnant", "glucose", "pressure", "triceps",
    "insulin", "bmi", "pedigree", "age", "diabetes"
  )
)
```